

PA4: Beat the Interpreter

CS6004: Code Optimization for OO Languages

April 18, 2026

1 Optimization Description

1.1 What We Implemented

We built two complementary OO optimizations on top of a from-scratch Andersen-style interprocedural pointer analysis:

(1) PTA-Driven Method Inlining. Inlining replaces a call site with the body of the callee, getting rid of dispatch overhead and opening the door for further optimizations on the inlined code. We handle two categories:

- *Static calls* — always safe to inline; no dispatch ambiguity.
- *Virtual/interface calls* — inlined only when our PTA proves $|pts(r)| = 1$, meaning exactly one concrete type ever reaches the receiver at that site.

The pass runs iteratively up to five times, because inlining one method can reveal new call sites in the inlined body that were not visible before — for instance, inlining `process()` exposes `execute()`, which a second pass can then eliminate.

(2) Null-Check Elimination. Soot inserts an explicit null-guard before every virtual call and field access in Jimple IR. When our PTA proves $pts(r) \neq \emptyset$, that guard is dead code, so we swap the `IfStmt` out for a `NopStmt`. We deliberately run this pass *after* inlining: once a method is inlined, its allocation site becomes visible in the caller’s scope, making non-nullness provable for variables that would have looked opaque before (as in Test10).

1.2 Why It Is Useful

With JIT disabled (`-Xint`), every `invokevirtual` or `invokeinterface` instruction pays a full vtable/itable lookup on each iteration of a hot loop. Inlining turns those into plain arithmetic, eliminates call-frame overhead, and leaves the interpreter with a flat instruction stream to execute. Across our positive test cases we see 37–79% wall-clock improvement with an average of 54.7%. Null-check elimination adds to this: once allocation sites are brought into scope by inlining, the dead branches disappear and the loop tightens further.

2 Analysis Used

2.1 Andersen-Style Pointer Analysis

We implement a *flow-insensitive*, *context-insensitive*, *inclusion-based* interprocedural pointer analysis (Andersen 1994) over Soot’s Jimple IR. For every reference-typed local v , the analysis computes an over-approximation of $pts(v)$: the set of abstract heap objects (one per allocation site) that v could point to at any point in the program.

Constraints generated from Jimple:

Statement	Constraint	Effect
$v = \text{new } T()$	NEW	$pts(v) \supseteq \{h_{T,\ell}\}$
$v = u$	COPY	$pts(v) \supseteq pts(u)$
$v = (T) u$	CAST	$pts(v) \supseteq pts(u)$
$v = u.f$	LOAD	$\forall h \in pts(u) : pts(v) \supseteq fieldPts(h, f)$
$u.f = v$	STORE	$\forall h \in pts(u) : fieldPts(h, f) \supseteq pts(v)$
$r.m(\dots)$	CALL	resolve via $pts(r)$; bind args and return

The call graph is constructed on the fly: we seed the worklist with `main`, discover new callees as receiver points-to sets grow, and keep iterating until nothing changes.

2.2 Dimensions of Precision

- *Flow-insensitive*: we compute a single points-to set per variable rather than one per program point. This is an over-approximation, but it is tractable and works well for the OO patterns we care about, where variables are typically assigned once and used inside a hot loop.
- *Context-insensitive*: all calls to the same method share one constraint set. Points-to sets from different call sites may get merged, but the analysis stays tractable.
- *Field-sensitive*: we maintain a separate $fieldPts(h, f)$ for each heap object h and field f . This is what makes the strategy pattern work (Test7) — without it, we cannot track which concrete type sits in the `strategy` field of a specific `Processor` object and so cannot prove `execute()` is monomorphic.
- *Heap abstraction*: one abstract object per allocation site (method $\times$ statement). In Test1, for example, two separate `new` expressions give us $pts(a) = \{h_{Dog}\}$ and $pts(b) = \{h_{Cat}\}$ as independent sets, so both calls can be inlined without interfering with each other.

2.3 Constructor Seeding

For field stores inside `<init>` to be handled correctly, `this` inside the constructor must be bound to the right heap object rather than left as an unresolved formal. We handle this by eagerly enqueueing a class’s constructor the moment a `new T()` site is discovered, and by emitting `CopyEdge(receiver, thisParam)` for every `specialinvoke <init>`. This ensures STORE constraints inside constructors flow into the correct $fieldPts$ entry. Without this fix, Test7’s `strategy` field stays empty and `execute()` never gets inlined.

2.4 Assumptions

We assume no reflection, dynamic class loading, or concurrent writes. We also conservatively skip inlining whenever the target class has private members and the call crosses a class boundary. Java 11+ bytecode emits direct `getField/putField` on private fields and relies on `NestHost` attributes to authorize the access; Soot’s ASM backend silently drops those attributes, which would cause `IllegalAccessError` at runtime. For test cases where cross-class inlining is needed, we simply make the relevant fields package-private instead (Tests 5, 7, 10).

3 Transformation Details

3.1 Before and After: Virtual Call Elimination (Test5)

Our PTA establishes $pts(c) = \{h_{Counter}\}$, so both `increment()` and `get()` are monomorphic and get inlined.

Before inlining — each loop iteration goes through a virtual call:

```

1 label1:
2   if i1 >= 1000000 goto label3;
```

```

3   virtualinvoke $r0.<Counter: void increment()>();
4   i1 = i1 + 1;
5   goto label1;
6 label3:
7   $i0 = virtualinvoke $r0.<Counter: int get()>();

```

Listing 1: Original hot loop with virtualinvoke

After inlining — the virtual dispatch is gone; the loop now directly reads and writes the field:

```

1 label1:
2   if i1 >= 1000000 goto label3;
3   $i2 = $r0.<Counter: int value>;
4   $r0.<Counter: int value> = $i2 + 1;
5   i1 = i1 + 1;
6   goto label1;

```

Listing 2: After inlining: no virtual dispatch in loop

3.2 Before and After: Two-Level Inlining (Test7)

Pass 1 inlines `process()` into `main`, at which point a call to `strategy.execute()` becomes visible. Pass 2 picks this up: field propagation through the `Processor` constructor establishes $pts(\$r_5) = \{h_{\text{Multiply}}\}$, so `execute()` is inlined as well and only the multiplication remains:

```

1   $r5 = r4.<Processor: Strategy strategy>;
2   $l11 = 19 * l10;    // execute() body inlined: return a * b

```

Listing 3: After two passes: interface call replaced by multiplication

3.3 Key Challenges

SiteInliner preconditions. Soot’s `SiteInliner.inlineSite()` silently corrupts the IR when the call site is not a bare `InvokeStmt` or a top-level `AssignStmt` — for example, when an invoke expression appears as an argument to another call. We guard against this with an `isCallSiteInlinable()` check, and wrap every inline attempt in a `body.validate()` call so we can roll back cleanly if something goes wrong.

Recursion safety. We keep a set of `(container, target)` pairs seen within the current pass. Without this, inlining `fib()` creates a fresh copy of its body that contains more `fib()` calls, which would be inlined again in the same scan — and so on forever. The deduplication set breaks that cycle. Test9 confirms the optimizer terminates and produces the right answer.

4 Test Cases

We wrote 10 test cases, each targeting a specific aspect of the pipeline. Tests 8 and 9 are intentional negative cases (the specification allows at most 3).

- **Test1** — Two distinct monomorphic receivers (`Dog` and `Cat`) in the same hot loop. Checks that the PTA keeps $pts(a)$ and $pts(b)$ separate and inlines both virtual calls independently.
- **Test2** — Field populated through a setter rather than a constructor. Exercises the STORE constraint in a regular virtual call and checks that the subsequent LOAD in `execute()` correctly resolves to a singleton set.
- **Test3** — Four-level static call chain (`f1`–`f4`). The inner method-changed loop collapses all four levels into flat arithmetic within a single pass, giving the highest improvement at 79.3%.

- **Test4** — Only `SubA` is ever allocated, even though the class hierarchy has two subclasses. CHA would conservatively see two possible targets and refuse to inline; our PTA correctly narrows $pts(b) = \{h_{\text{SubA}}\}$ and inlines.
- **Test5** — Counter with a package-private `value` field. Checks that both `increment()` and `get()` get inlined and that the `NestHost` / private-member restriction is handled correctly.
- **Test6** — Object allocated inside a factory method. The PTA must follow the return value across the method boundary to conclude $pts(s) = \{h_{\text{Widget}}\}$.
- **Test7** — Strategy pattern with a field-stored strategy. Two-pass inlining: `process()` in Pass 1 reveals `execute()`, which is inlined in Pass 2.
- **Test8** — *Negative*. The receiver genuinely alternates between `Circle` and `Rectangle` at runtime, so $|pts| = 2$ and the optimizer must not inline. Confirms soundness when the call site is truly polymorphic.
- **Test9** — *Negative*. Recursive `fib()`. The deduplication guard prevents infinite unrolling; the optimizer terminates and produces the correct answer.
- **Test10** — Both optimizations working together. Inlining `makeNode()` brings its allocation into `main`'s scope; the null-check pass can then remove the dead guard before `compute()`, which is also inlined.

5 Results

5.1 Functional Correctness

All 10 tests pass output comparison between the original and optimized class files under `-Xint`. The two negative tests confirm that the optimizer recognises polymorphic and recursive sites and leaves them untouched.

Tests	Result
Test1–Test7, Test10	PASS
Test8, Test9 (negative)	PASS (no transformation, output unchanged)

5.2 Performance (Wall-Clock, `-Xint`, 5-run average)

Test	Original (ms)	Optimized (ms)	Delta (ms)	Improvement
Test1	126	48	78	61.9%
Test2	120	37	83	69.2%
Test3	232	48	184	79.3%
Test4	50	30	20	40.0%
Test5	74	35	39	52.7%
Test6	78	49	29	37.2%
Test7	91	40	51	56.0%
Test10	229	134	95	41.5%
Avg (positive tests)				54.7%
Test8 (negative)	187	196	−9	−4.8%
Test9 (negative)	699	714	−15	−2.1%

Test3 comes out on top at 79.3%, where four levels of static calls collapse into a single flat loop body in one pass. Test2 and Test1 are close behind at 69.2% and 61.9% — interface dispatch on a million-iteration loop is expensive, and eliminating it entirely shows clearly in the numbers. Test10 is a good illustration of why we run both passes in sequence: inlining `makeNode()` sets

things up so the null-check pass can fire, and together they give 41.5%.

Tests 8 and 9 regress slightly ($<5\%$), which is just the cost of running the PTA and Soot pipeline with nothing to show for it. This is expected. The average of 54.7% across positive tests comfortably exceeds the 3–5% threshold for real benchmarks cited in the assignment specification.